2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3315-7434-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/COMPSAC65507.2025.00169

MVGText: Momentum and Variance Guided Hard-Label Text Test Case Generation

Shunhui Ji, Yu Shi, Changrong Huang, Letian Cheng, Pengcheng Zhang*
Key Laboratory of Water Big Data Technology of Ministry of Water Resources, Hohai University, Nanjing, China
College of Computer Science and Software Engineering, Hohai University, Nanjing, China
{shunhuiji, shy, changronghuang, letiancheng, pchzhang}@hhu.edu.cn

***Abstract*—Deep neural network (DNN) based text intelligence softwares have proliferated and have been widely used. However, they face challenges such as robustness. Testing is essential to DNNs in real-world applications. Researchers have proposed various testing techniques for generating test cases. With the black-box hard-label setup where only the output label of the DNN is accessible, related studies mainly concentrated on generating the new text test case by iteratively perturbing single initial test case, which limits the effectiveness. To address this issue, this paper proposes a new text test case generation method MVGText (Momentum and Variance Guided hard-label Text test case generation) for text-oriented DNN, which generates high-quality text test cases by searching based on multiple initial test cases. Firstly, multiple initial text test cases are generated randomly, in which global fixed word replacement is employed to enhance the success rate. Then, momentum and variance are applied to guide the search for the more imperceptible test case. Finally, greedy optimization is used to obtain a higher quality test case. Experimental evaluation shows that MVGText achieves an average post-test success rate of 1.4%. Furthermore, the test cases generated by MVGText exhibit the highest similarity to the original text compared to other comparative methods.**

***Index Terms*—Text Software, Test Case Generation, Multiple Cases, Momentum and Variance, Greedy Optimization**

I. Introduction

With the rapid development of deep neural network (DNN), a series of intelligent software based on the DNN have appeared and have made remarkable achievements in many fields such as computer vision (CV) [1], natural language processing (NLP) [2], speech recognition [3], and so on. However, existing DNNs have robustness pitfalls and are susceptible to being misled by artificially purposely constructed test cases [4] [5] and outputting incorrect judgments. A test case is a type of input designed to validate a specific function or behavior of a software system, where cases are generated by adding small perturbations to the raw data [6]. High-quality test cases are able to trick DNN into generating false predictions while maintaining semantic or visual consistency that is virtually imperceptible to humans.

In the field of image intelligent software testing, test case generation techniques are relatively mature, especially the gradient-based generation method [7]. Unlike image data, text data are composed of discrete symbols, and this discrete nature limits the study of text test case generation [8] [9]. How to introduce the test case generation techniques for image intelligence software to text is an issue of interest [10].

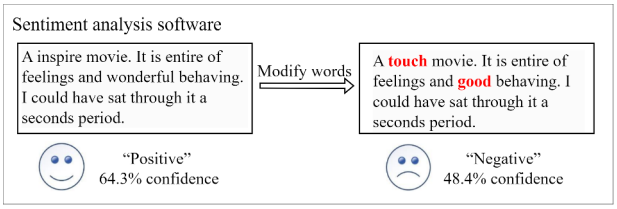

Fig. 1. An Example of Testing Text Intelligence Software.

Fig. 1 shows how the robustness of sentiment analysis software can be tested by modifying words in the text to generate a test case. The original text was classified as "Positive" sentiment with 64.3% confidence. Replacing only two words in the text (replacing "inspire" with "touch", "wonderful" with "good"), the modified text was incorrectly categorized as "Negative" sentiment with 48.4% confidence. This test shows that text-based intelligence software such as sentiment analysis is very sensitive to specific words in text, and intentionally constructed test cases can mislead the intelligence software and expose the model's shortcomings in robustness. Therefore, an in-depth study of test case generation techniques is of great significance to improve the defense capability of models and ensure the robustness of intelligent software.

Test case generation techniques are categorized into white-box generation techniques [11] [12] and black-box generation techniques [13] [14] based on the degree of access to information about the model under test. White-box generation techniques involve exhaustive knowledge of the internal structure and parameters of the test model, while black-box generation techniques are limited to accessing the output of the model, including labels and its confidence scores. Further, the black-box generation technique can be subdivided into soft-label [14] and hard-label [8], wherein the hard-label method does not obtain the confidence scores of the model output. The black-box hard-label method has high practical value because it does not rely on the internal structure of the model and confidence scores, and is closer to the actual application scenario [15].

However, existing methods for generating hard-label test cases are limited to optimizing the single initial case, which

*Corresponding Author.

2836-3795/25/$31.00 ©2025 IEEE
DOI 10.1109/COMPSAC65507.2025.00169

makes it difficult to utilize the beneficial information in the already generated test cases and affects the similarity of the generated test cases to the original text. Before searching for a new case, a test case generated by random initialization switches back to the original word to reduce the number of word substitutions, and therefore may miss more important positions that need to be replaced.

In image-related tasks, computing the momentum and variance across multiple samples [16] can effectively guide the search process, thereby providing directional insights for the generation of text test cases.

Based on this, we propose a momentum and variance guided hard-label text test case generation method, utilizing multiple cases for search, named MVGText [1]. In the initialization phase, we add global fixed word replacement to make the initialization richer for multiple test cases. In the search phase, we refer to the momentum and variance in the image domain [16]. We use information from multiple cases to guide the future search process, using noise-adding and noise-reducing operations to make the search process more stochastic. Since it is not possible to obtain the gradient in a hard-label setting, we design a data structure and searched using the perturbation matrix. We also design a quality score balance perturbation and similarity. In the optimization phase, we perform greedy synonym replacement strategy on the current best test case to generate higher quality cases. The main contributions of this paper are as follows.

- We propose a new test case generation method, named MVGText, which use the momentum and variance based on multiple initial test cases to guide the generation of new test cases.
- We optimize the initialization process to improve the success rate of the initial test cases. And we perform greedy optimization to further improve the quality of test case.
- The experimental results show that MVGText has an average post-test success rate of 1.4% on all eight datasets and generates test cases with the best similarity to the original text. The ablation study validated the effectiveness of the three improved methods.

The rest of the paper is organized as follows. Section II describes the related work of this study. Section III provides a detailed description of the proposed MVGText. Section IV shows the experimental setting and Section V shows the results and analysis. Finally, the paper is summarized in Section VI.

II. Related Work

Text test case generation methods can be classified into white-box and black-box according to their access to the test model, and black-box can be classified into soft-label and hard-label according to whether they can access confidence scores.

[1]The codes are available at https://github.com/satatata-ai/MVGText.

A. White-Box

Samanta et al. [17] proposed a gradient-based method to design test cases, showing its effectiveness in misleading text classification models in various natural language processing tasks. Sato et al. [18] propose an interpretable perturbation method in the input embedding space for text, which generates test cases by perturbing word embeddings while preserving semantic coherence, enabling both effective attacks and human-understandable modifications. Behjati et al. [19] propose a universal test method for text classifiers, which generates input-agnostic perturbations in the embedding space to fool a wide range of models, demonstrating the vulnerability of text classifiers to universal test cases. Zhang et al. [12] propose a method for generating fluent test cases in natural language by leveraging gradient-based optimization and language model scores, ensuring the effectiveness of testing and language fluency. Zheng et al. [20] propose a study on test cases in dependency parsing, demonstrating that parsers are vulnerable to adversarial perturbations at both sentence and phrase levels, and further show that incorporating high-quality test cases during training can improve model robustness without sacrificing performance on clean data. Meng et al. [21] propose a geometry-inspired test method for generating natural language test cases by iteratively approximating the decision boundary of DNN, achieving high success rates with minimal word replacements while maintaining human-imperceptible perturbations.

B. Black-Box

Black-box, which do not have access to internal information, are more versatile than white-box. The soft-label in a black-box gets labels and confidence scores, and the hard-label only gets labels. Soft-label methods that can be subdivided into character level, word level, sentence level and multi-level. Hard-label methods concentrate on replacement of words.

Soft-label. Character level means, operations such as adding, deleting, modifying or swapping characters. Gao et al. [22] propose DeepWordBug, a black-box test algorithm that generates small text perturbations by identifying and modifying the most important words using character-level transformations, significantly reducing the accuracy of deep learning classifiers on real-world text datasets. Word level is the manipulation of words. Jin et al. [23] propose TextFooler, a strong and efficient baseline for generating test cases that effectively fools models while preserving semantic content, syntax, and human readability, with linear computational complexity. Li et al. [24] propose a novel framework, Adversarial Text Generation by Search and Learning (ATGSL), which treats black-box text test cases generation as an unsupervised text generation problem and introduces three methods to generate high-quality test cases with improved semantic similarity and attack efficiency. The sentence level is used to modify or add sentences. Jia et al. [25] propose an evaluation scheme for the Stanford Question Answering Dataset (SQuAD) to test the true language understanding

abilities of reading comprehension systems by introducing adversarially inserted sentences, revealing a significant drop in model performance and highlighting the need for more precise language understanding models. Multi-level means that test cases are generated using two and more levels of granularity. Xu et al. [26] propose PromptAttack, an efficient prompt-based test tool that audits the robustness of large language models (LLMs) by generating test cases through a composed attack prompt, achieving higher success rates compared to benchmarks, while maintaining semantic fidelity through a filtering mechanism.

Hard-label. Maheshwary et al. [8] propose a method for generating hard-label test cases based on generative algorithms, abbreviated as HLGA. However, HLGA requires a large number of queries due to the need for numerous test case candidates and is prone to falling into local optima. Ye et al. [27] propose TextHoaxer, which formulates the task of hard-label test case generation as a perturbation matrix based gradient optimization problem in a continuous word embedding space. Since the text space is discrete, the gradient estimation may be inaccurate and the quality of the generated test cases may be severely affected. Ye et al. [9] propose LeapAttack, the main improvement of LeapAttack is the introduction of iterative round trips between discrete candidates and continuous gradients, using the word embedding space as an auxiliary space as a bridge, allowing a candidate to move freely through the embedding space. Existing methods suffer from high complexity of genetic algorithms and inaccurate gradient estimation. Liu et al. [28] propose SSPAttack, which first initializes a test case, then determines the order of replacement and searches for anchor synonyms, thus avoiding traversing all synonyms. Ye et al. [29] propose PAT, which explicitly models adversarial and non-adversarial prototypes. PAT is able to select better candidate words for the perturbation location in a geometry-aware manner. Liu et al. [30] propose HyGloadAttack, which is a hybrid optimization strategy that designs strategies to filter potentially low-quality cases while searching quickly in the word embedding space.

Existing hard-label test case generation methods that only initialize a single case for the next search may miss higher quality test cases. There are limitations in using only embedding space optimization or design strategy optimization. T-PGD [10] applies PGD [31] methods for images to text, which requires proxy models that are costly. Text hard-label test case generation needs to be explored further.

III. Method

A. Problem Formulation

Suppose there exists an original text X consisting of n words, $X = \{w_1, w_2, ...w_n\}$, and the ground truth label of X is y. There exists a test model f such that $f(X) = y$, and our task is to generate X' by replacing w_i, such that $f(X') \neq y$. X' is the test case that can mislead the test model. The difference between X' and X in the word embedding space is called the perturbation matrix. High quality text test cases need to maximize semantic similarity while ensuring test success, as formally represented by the following equation.

$$X_{test} = arg \max_{X'} Sim(X, X'), s.t. f(X') \neq f(X) \quad (1)$$

where $Sim(X, X')$ is the similarity between the original text X and the test case X', and X_{test} is the one test case with the highest similarity among all X'.

B. Overview

To fully utilize the information from the generated test cases, we propose a momentum and variance guided hard-label text test case generation method, named MVGText. The overview of MVGText is shown in Fig. 2, which consists of three steps.

Step 1: Initialization. Firstly, the input text is preprocessed. After randomly initializing a test case, global fixed word replacement is performed which can help improve test success rate. Finally, α test cases are obtained.

Step 2: Search for the new test case. In order to find the test case with lower perturbations, the momentum and variance are calculated for the current best test case to guide the search. Then, momentum and variance based noise is added and random noise is reduced to increase the randomness. With the noise information of the success test cases, synonym replacement is performed following by changing back to the original word to get a better test case.

Step 3: Greedy optimization. For the best test case found so far, the changed words are identified as the words to be optimized. By updating these words with all the synonyms, all the optimization candidates can be obtained. Then the similarity between each optimization candidate and the original sample is calculated and the better candidates are retained. By sorting these candidates in descending order, the best one that can mislead the target model is selected as the final test case.

The full process of MVGText is described in *Algorithm* 1.

C. Initialization

We designed three data structures for storing test cases, failure cases and update test cases, called $Success$, $Fail$ and $Update$ respectively.

Preprocess the data. The dataset is first loaded and preprocessed, then pre-trained test models, pre-trained word embedding models (e.g., counterfitted word vectors [32]) are loaded and a set of synonyms is constructed. Data preprocessing includes tokenization, removal of stop word, etc. Tokenization is the process of segmenting text into meaningful lexical units (tokens), while stop word removal eliminates high-frequency, low-information words (e.g., "the", "is") to enhance computational efficiency and semantic relevance in text processing.

Initialize randomly. The test success rate of a test case is directly related to the initialization, which is affected by the random seed. For better comparison, we set the same random seed as existing methods and add global fixed

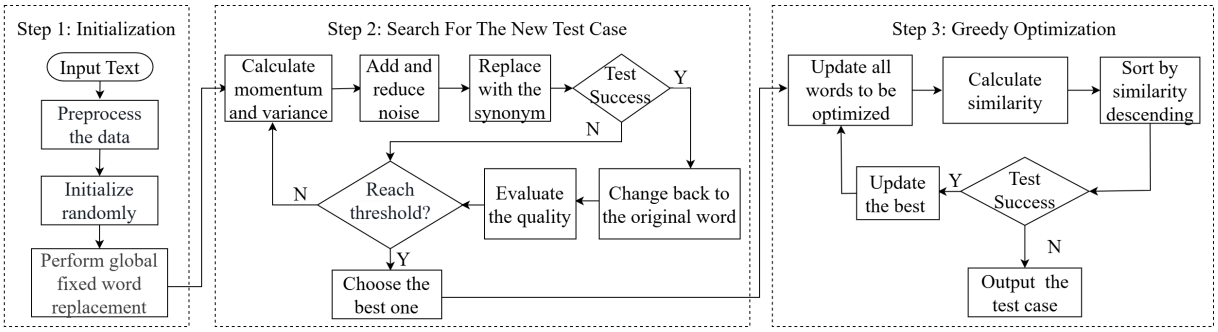

Fig. 2. Overview of MVGText.

word initialization after the random initialization strategy of HLGA. This ensures the uniqueness of the output.

Random initialization includes local word replacement and global word replacement. The core idea of local word replacement is to generate a test case by replacing some words in the text. After each replacement, the test model is queried to determine if it is a test case, if so, MVGText calculates its semantic similarity to the original text and adds the generated test case and similarity to $Success$; otherwise, it is added to $Fail$. This process ensures that valid test cases are found within a limited number of attempts by incrementally increasing the range of replacements and the number of queries. If the local word replacement does not find a test case, the global word replacement is performed, otherwise, the global word replacement is skipped. Unlike local word replacement, global word replacement randomly selects words for replacement across the entire text range, limiting the query test model to no more than 2,500 times. The lexical categories that limit the words to be replaced are adjectives, adverbs, verbs, and nouns.

Perform global fixed word replacement. Global fixed word replacement means that, a fixed-index synonym substitution is performed at every word position that satisfies the lexical constraints. There are 49 synonyms of the original word in the synonym set, indexed from 1 to 49. 49 cases can be generated by performing synonym substitution on all replaceable positions using these 49 indexed positions in turn. For example, the first case is generated by replacing all the positions to be replaced with the corresponding synonyms with index 1. Test them to see if they are test cases until the initialization is complete after the α test case is generated. The worst case scenario is that none of the 49 cases generated are test cases. If the initialization does not generate a test case, the task of generating a test case for this text fails. The generated α initial test cases will be used for the search in step 2.

Adding global fixed word replacement increases the success rate of the initialization, thus finding more test cases based on the original test success rate. there is a correlation between the choice of α and the subsequent optimization, as will be shown in Section V-A. We switched back to the original word one by one in all the test cases of the initialization to find test cases with fewer perturbed words while guaranteeing the success of the test. Three data structures were updated synchronously. The α test cases with the highest similarity are stored in $Update$ in descending order of similarity. The initialization steps are described in lines 1-3 of *Algorithm* 1.

D. Search for the New Test Case

Existing approaches focus on optimization using a single test case, ignoring the possibility that higher quality test cases may exist. We refer to the image test case generation approach [16], which uses information from multiple test cases to calculate momentum and variance based on the current test case with the highest similarity, X_{best}, to guide the generation of new test cases.

Calculate momentum and variance. Due to the discrete nature of text, gradient-based methods in the image domain are difficult to apply directly to text. We take the variation of the text in the word embedding space as the gradient, take the test cases in $Update$ as the neighbors of X_{best}, and compute the difference between X_{best} and the original text X_{ori}, i.e., the $Noise_{best}$ (momentum). Calculate the difference between all test cases in $Update$ and the original text in the word embedding space and sum the average (variance), plus the value from the last iteration to the current cumulative variance $Noise_{cur}$.

Add and reduce noise. To increase the randomness of the search process, we designed the noise-adding and noise-reducing processes (*Algorithm* 1, line 8 and line 12). Based on the momentum $Noise_{best}$ and cumulative variance $Noise_{cur}$ calculated in the previous step, $Noise_{best}$ and $Noise_{cur}$ are weighted to obtain the $Noise_{new}$ with the following equation.

$$Noise_{new} = \mu \times Noise_{best} + (1 - \mu) \times Noise_{cur} \quad (2)$$

μ is a random number from 0.5 to 1.

Reduce a random noise in $Noise_{new}$ to get the final noise $Noise_{new}$.

Replace with the synonym. We first use the noise obtained in the previous step to determine the location of the synonym replacement. For each word in the position where the noise is not zero, calculate the original word vector plus

the word noise corresponding to $Noise_{new}$ to get the target perturbation vector, traverse all the synonyms of the word, calculate the Euclidean distance between the synonym vector and the target perturbation vector, and find the synonym with the smallest distance as the best synonym. Finally, the corresponding word in the original text is replaced with the best synonym to complete the case update. This process ensures that the replacement word is semantically close to the original word, and at the same time exploits the noise information of successful test cases to enhance the probability of successful tests. The updated cases are then checked to see if they are test cases. Before each query of the test model, it is necessary to check whether the test case appears in $Success$ or $Fail$, this step is to avoid repeated queries.

Change back to the original word. If the test case is updated, the quality of the test case is further improved by switching as many of the replaced words in the test case back to the original words as possible. For the current test case, we gradually switch the replaced words back to the original words. When choosing the reduction target, under the condition of ensuring that the reduced cases can still be tested successfully, we consider all possible reduction options separately, and sort the samples by semantic similarity after the reduction respectively, and repeat this operation with the highest semantic similarity as the currently determined reduction to get a test case with as little perturbation as possible, and save it to $Success$.

Evaluate the quality. To balance the perturbation rate and similarity of words, we design the quality score of test cases as "$Perturbation-Similarity$", and the smaller the quality score, the better, which represents the less perturbation of words and the higher similarity. We evaluate the quality of all test cases in $Success$ and sort them, and select the top α test cases to update $Update$.

Judge the threshold. We update $Update$ in the inner loop with its maximum number of iterations γ set to 20. Considering that the momentum is relatively stable, we update it in the outer loop. If α test cases in $Update$ remain unchanged for 40 consecutive times, or the outer loop reaches the maximum number of iterations 20, the search ends. It is worth noting that if the initialized test cases replace only one word, we skip the search phase to reduce the number of queries. Lines 4-16 of *Algorithm* 1 describe Step 2.

Algorithm 1 MVGText

Input: X_{ori}: Original text, f: Test model.
Output: X_{test}: Test case.

Step 1: Initialization
1: Initialize $Success$, $Fail$, $Update$;
2: Random initialization, global fixed word replacement;
3: Back to the word and get X_{best}, Sim_{best}, $Update$;
Step 2: Search for the new test case
4: **while** the threshold is not reached **do**
5: Initialize $Noise_{best}$, $Noise_{cur}$;
6: **for** $i = 1$ to γ **do**
7: **for** $X_j \in Update$ **do**
8: $Noise_{cur} = Noise_{cur} + Sub(X_j, X_{ori})$;
9: **end for**
10: $Noise_{cur} = Noise_{cur}/\alpha$;
11: $Noise_{new} = \mu \times Noise_{best} + (1-\mu) \times Noise_{cur}$;
12: $Noise_{new} = Noise_{new} - random$;
13: Use $Noise_{new}$ to find synonyms and replace;
14: Update $Success$, $Fail$, $Update$;
15: **end for**
16: **end while**
Step 3: Greedy optimization
17: Choose the best in $Update$ as X_{best}, get Sim_{best};
18: X_{ori} and X_{best} different words into $Choices$;
19: **while** a new test case is gotten **do**
20: **for** $word \in Choices$ **do**
21: **for** $syn \in Synonyms(word)$ **do**
22: Replace $word$ with syn, calculate the Sim;
23: When $Sim > Sim_{best}$, deposit $Replaced$;
24: **end for**
25: **end for**
26: **if** $len(Replaced) > 0$ **then**
27: $Replaced$ descending order, query f;
28: **if** a new test case is gotten **then**
29: Update X_{best} and Sim_{best}, X_{test} = X_{best};
30: **end if**
31: **end if**
32: **end while**
33: **return** X_{test}

E. Greedy Optimization

For the optimal test case X_{best} obtained in Step 2, we perform a greedy optimization as in Lines 17-32 of *Algorithm* 1.

Update all words to be optimized. For each word in the X_{ori}, if it is different from the corresponding word in X_{best}, the word is marked as a word to be optimized, and a set of words to be optimized $Choices$ is obtained. for each word to be optimized, it is updated with all the synonyms, and optimization candidates are obtained. Suppose there are n words to be optimized and each word has 49 synonyms, then $n \times 49$ optimization candidates are obtained.

Calculate similarity. For each candidate of X_{best} and calculate its semantic similarity to X_{ori}. If the similarity of the candidate is higher than the current best similarity, the candidate is added to the list $Replaced$.

Sort by similarity descending. Sort the candidate cases in $Replaced$ in descending order of similarity and check one by one whether they can successfully mislead the target model, if yes, update X_{best}. if none of them can, end the optimization search.

IV. Experimental Setting

A. Experimental Environment

All experiments are performed on an Ubuntu 18.04.6 LTS server with an NVIDIA RTX 3090 24G GPU using Python

3.7, the dataset and test model provided by [8].

B. Datasets and Models

To demonstrate the generality of the proposed method, we use five classification datasets and three natural language inference (NLI) datasets to conduct experiments on three pre-trained test models. Each dataset consists of 1000 samples.

The five classification datasets are as follows.

1) MR [33] is a movie review dataset for sentiment analysis containing positive or negative categories. The average length is about 20.

2) AG [34] is a classification dataset of news in four categories, world, sports, business and science.

3) Yahoo [34] is a question-and-answer dataset, constructed for topic classification and includes ten categories.

4) Yelp [34] is a document-level sentiment classification dataset consisting of two categories.

5) IMDB [35] is a document-level movie review dataset consisting of two categories with longer reviews than MR.

The three NLI datasets are as follows. The average lengths of all NLI datasets are less than 15.

6) SNLI [2] is a dataset for natural language inference, from Amazon Mechanical Turk. Labels include entailment, neutral and contradiction.

7) MNLI [36] is one of the more extensive dataset used to study natural language inference. There are three labels.

8) mMNLI [36] is a variant of the MNLI characterized by the inclusion of mismatched pairs of premise hypotheses.

We use Long Short-Term Memory (LSTM) [37], Convolutional Neural Network (CNN) [38], and Bidirectional Encoder Representations from Transformers (BERT) [39] models as the test models. These models are pre-trained on five classification datasets. The BERT model is also pre-trained on three NLI datasets. The model parameters are adopted from the study of [8]. The original classification success rates (Acc_ori) of the three tested models on the five classification datasets are shown in Table I. Testing BERT on the three NLI datasets, Acc_ori is 89.10% for SNLI, 85.10% for MNLI, and 82.10% for mMNLI.

TABLE I
ACC_ORI FOR CLASSIFICATION DATASETS.

Acc_ori	MR	AG	Yahoo	Yelp	IMDB
LSTM	78.00%	90.20%	73.70%	94.80%	89.30%
CNN	76.50%	90.40%	71.10%	92.90%	87.80%
BERT	85.00%	93.00%	79.40%	95.30%	87.50%

C. Comparative Methods

We have selected several recent hard-label test case generation methods for comparison.

1) HLGA [8]. A representative heuristic-based algorithm, minimizing the search space and optimizing the semantic similarity of test cases using genetic algorithms.

2) TextHoaxer [27]. It formulates the problem as a gradient-based optimization of the perturbation matrix in the word embedding space.

3) LeapAttack [9]. It is also a perturbation matrix-based test case generation method, with the main improvement being the introduction of iterative round trips between discrete candidates and continuous gradients.

4) PAT [29]. Explicitly modeling adversarial and non-adversarial prototypes to select candidate words in a geometry-aware manner.

5) HyGloadAttack [30]. A hybrid optimization strategy that utilizes perturbation matrix optimization in the word embedding space while designing strategies to filter low-quality cases.

D. Evaluation Metrics

To better assess the quality of the hard-label test cases generated by the proposed method, we choose the following evaluation metrics.

1) Semantic similarity (Sim↑). Sim is an important metric to evaluate the quality of test cases, the validity of test cases depends on whether the original semantics can be preserved and tested successfully. We use a universal sentence encoder [40] to calculate the semantic similarity, the higher the similarity, the better.

2) Post-test accuracy (Acc↓). The performance of test method is measured by the test model's accuracy for the generated samples, referred to as post-test accuracy. This accuracy indicates the effectiveness of the test method, with lower values implying superior capabilities. N_{fail} denotes the number of failed samples, and N_{total} denotes the number of all samples.

$$Acc = \frac{N_{fail}}{N_{total}} \times 100\% \tag{3}$$

3) Word perturbation rate (Pert↓). Pert denotes the average ratio of the number of replaced words to the total number of words in the original text for the generated test cases compared to the original text, where n_{X_i} denotes the number of word modifications of text X_i and $len(X_i)$ denotes the total number of words in text X_i, N_{suc} denotes the number of cases that are successfully tested.

$$Pert = \frac{1}{N_{suc}} \sum_{i}^{N_{suc}} \frac{n_{X_i}}{len(X_i)} \times 100\% \tag{4}$$

4) Queries for the model (Qrs↓). Qrs is used to evaluate the average number of times a test case generation method needs to query the test model when generating test cases.

V. EXPERIMENTAL RESULTS AND ANALYSIS

In order to utilize multiple cases to search, the value of the parameter α needs to be determined. We have done parametric experiments at Section V-A to determine the α needed for subsequent experiments. Comparative experiments are done in Section V-B, and ablation experiments are utilized in Section V-C to demonstrate the advantages of the three proposed improvements.

A. Parameter Experiments

In order to save computational resources while experimenting as comprehensively as possible, three models are tested on one classification dataset MR. On three NLI datasets, test the BERT. The value of the parameter α is set to 1, 3, 5, 7, 9. We evaluated four metrics, Acc, Qrs, Pert, and Sim. The experimental results for MR are shown in Table II. Bold indicates that the metric is optimal on that dataset. It can be seen that Acc is independent of α, since Acc is only relevant to the success of the initialization of MVGText.

TABLE II
PARAMETER EXPERIMENT RESULTS ON MR.

Dataset	Model	Metric			
		Acc↓	Qrs↓	Pert↓	Sim↑
LSTM	α=1	0.70%	**181.3**	11.23%	0.753
	α=3	0.70%	285.9	10.54%	0.768
	α=5	0.70%	375.8	**10.35%**	0.770
	α=7	0.70%	437.9	**10.35%**	**0.773**
	α=9	0.70%	456.9	10.43%	**0.773**
CNN	α=1	0.60%	**182.7**	10.88%	0.763
	α=3	0.60%	290.7	10.34%	0.779
	α=5	0.60%	383.7	**10.08%**	0.783
	α=7	0.60%	436.8	10.29%	**0.784**
	α=9	0.60%	463.6	10.21%	**0.784**
BERT	α=1	1.00%	**197.2**	10.74%	0.753
	α=3	1.00%	295.6	10.07%	0.759
	α=5	1.00%	386.0	9.94%	0.763
	α=7	1.00%	435.3	**9.93%**	**0.765**
	α=9	1.00%	469.9	9.94%	0.764

The number of queries is positively correlated with α, using more test cases in the search process increases the cost of the experiment. When $\alpha = 1$, it is similar to the previous method of optimization using a single case. As α increases, the Pert starts to decrease and the Sim starts to increase, and on the MR dataset, testing different models, $\alpha = 7$ achieves the best similarity, both $\alpha = 5$ and $\alpha = 7$ achieve relatively low perturbation rates.

The experimental results for the parameter α of NLI are shown in Table III, with a more obvious pattern. In the same way that the MR dataset demonstrated results on the different test models, the three NLI datasets tested the BERT, with Acc independent of α and Qrs positively correlated with α. Unlike the MR dataset, in NLI datasets, the perturbation rate decreases and the similarity increases as α increases. This may be related to the size of the dataset. The average length of all three NLI datasets is less than MR.

We roughly counted the consumption time and found that the value of α is positively correlated with the time required for test case generation, the larger the α, the longer the time required. Since the size of MR dataset is the smallest among the five classification datasets, combining the above, in order to control the time cost, we compromised by choosing $\alpha = 5$ for the next experiments.

TABLE III
PARAMETER EXPERIMENT RESULTS ON NLI.

Dataset	Model	Metric			
		Acc↓	Qrs↓	Pert↓	Sim↑
SNLI	α=1	1.10%	**105.3**	16.18%	0.561
	α=3	1.10%	146.4	15.79%	0.567
	α=5	1.10%	176.1	15.80%	0.575
	α=7	1.10%	198.5	15.74%	0.580
	α=9	1.10%	218.4	**15.71%**	**0.582**
MNLI	α=1	2.90%	**180.8**	12.35%	0.671
	α=3	2.90%	232.4	11.92%	0.683
	α=5	2.90%	273.0	11.72%	0.691
	α=7	2.90%	299.2	11.76%	0.694
	α=9	2.90%	314.6	**11.71%**	**0.695**
mMNLI	α=1	1.70%	**154.0**	11.53%	0.677
	α=3	1.70%	206.7	11.16%	0.688
	α=5	1.70%	257.0	11.21%	0.700
	α=7	1.70%	287.1	11.16%	0.700
	α=9	1.70%	305.9	**11.14%**	**0.703**

B. Comparative Experiments

Sim on classification datasets. In order to assess the quality of the generated test cases, we selected five baseline methods to compare the similarity. Among them, HLGA and PAT do not use perturbation matrix optimization. The experimental results are shown in Table IV, where our proposed MVGText has the highest similarity, which is attributed to the fact that our greedy strategy finds better quality test cases. Among the five baseline methods, HyGloadAttack exhibits the best similarity. PAT and LeapAttack perform worse than HyGloadAttack and have a large gap compared to MVGText. Testing the CNN on the IMDB dataset, we obtained the highest similarity of 0.954.

Acc on classification datasets. We also evaluated the Acc of MVGText on classification datasets. We chose HLGA as the base comparison method, TextHoaxer follows the HLGA setup, and HyGloadAttack is one of the most recent methods for generating text test cases for hard-label. The experimental results are shown in Table V.

It can be seen that overall MVGText performs the best, HyGloadAttack is optimized in the initialization phase compared to HLGA, but at the same time, it affects the raw test success rate, and gets the highest Acc only on a few tests. Compared to HLGA, we added global fixed word replacement, which ensures the raw test success rate while only by not exceeding 49 queries, and are able to find more test cases with improved Acc. TextHoaxer has the same initialization as HLGA, and the same Acc. HyGloadAttack is best on four tests, HLGA and TextHoaxer are best on eight tests, and MVGText is best on 13 tests.

Results on NLI datasets. We evaluated four metrics when testing BERT on three NLI datasets, Acc(↓), Qrs(↓), Pert(↓) and Sim(↑). To save computing resources, the experimental results compared to the advanced HyGloadAttack and LeapAttack are shown in Table VI.

MVGText has the lowest Pert and the highest Sim on all three datasets, the lowest Qrs on SNLI and mMNLI, and the

TABLE IV
COMPARATIVE EXPERIMENT RESULTS OF SIM.

Model	Dataset	Method					
		HLGA	TextHoaxer	LeapAttack	PAT	HyGloadAttack	MVGText(ours)
LSTM	MR	0.652	0.677	0.682	0.685	0.749	**0.770**
	AG	0.706	0.645	0.717	0.742	0.773	**0.785**
	Yahoo	0.691	0.677	0.699	0.703	0.765	**0.788**
	Yelp	0.831	0.808	0.850	0.867	0.879	**0.890**
	IMDB	0.901	0.891	0.916	0.922	0.932	**0.943**
CNN	MR	0.663	0.681	0.687	0.703	0.763	**0.783**
	AG	0.780	0.742	0.791	0.814	0.840	**0.845**
	Yahoo	0.761	0.750	0.783	0.800	0.841	**0.855**
	Yelp	0.838	0.813	0.862	0.880	0.888	**0.902**
	IMDB	0.913	0.903	0.927	0.936	0.940	**0.954**
BERT	MR	0.648	0.67	0.680	0.686	0.752	**0.763**
	AG	0.687	0.637	0.699	0.730	0.761	**0.763**
	Yahoo	0.707	0.698	0.710	0.724	0.781	**0.798**
	Yelp	0.775	0.739	0.799	0.819	0.839	**0.853**
	IMDB	0.866	0.853	0.882	0.893	0.912	**0.922**

TABLE V
COMPARATIVE EXPERIMENT RESULTS OF ACC.

Model	Dataset	Method			
		HLGA	TextHoaxer	HyGloadAttack	MVGText
LSTM	MR	0.70%	0.70%	**0.40%**	0.70%
	AG	5.70%	5.70%	**5.10%**	5.50%
	Yahoo	1.90%	1.90%	2.10%	**1.80%**
	Yelp	3.20%	3.20%	3.50%	**3.00%**
	IMDB	**0.30%**	**0.30%**	**0.30%**	**0.30%**
CNN	MR	**0.60%**	**0.60%**	0.70%	**0.60%**
	AG	1.40%	1.40%	1.70%	**1.30%**
	Yahoo	0.80%	0.80%	1.00%	**0.70%**
	Yelp	**0.60%**	**0.60%**	0.70%	**0.60%**
	IMDB	**0.00%**	**0.00%**	**0.00%**	**0.00%**
BERT	MR	**1.00%**	**1.00%**	1.10%	**1.00%**
	AG	2.60%	2.60%	3.50%	**2.00%**
	Yahoo	**0.40%**	**0.40%**	0.50%	**0.40%**
	Yelp	1.70%	1.70%	1.80%	**1.60%**
	IMDB	**0.00%**	**0.00%**	0.10%	**0.00%**

lowest Acc on MNLI and mMNLI. Overall, MVGText performs the best. MVGText has a lower perturbation rate and lower queries due to the use of momentum and variance to guide optimization during the search process using multiple samples.

MVGText has a better test success rate on most test models in both classification and NLI datasets, with an average post test success rate of 1.4%.

TABLE VI
COMPARATIVE EXPERIMENT RESULTS ON NLI.

Dataset	Model	Metric			
		Acc↓	Qrs↓	Pert↓	Sim↑
SNLI	LeapAttack	1.20%	267.9	15.95%	0.362
	HyGloadAttack	**1.00%**	197.5	15.87%	0.552
	MVGText(ours)	1.10%	**176.1**	**15.80%**	**0.575**
MNLI	LeapAttack	**2.90%**	298.3	11.95%	0.515
	HyGloadAttack	3.00%	**268.6**	11.88%	0.660
	MVGText(ours)	**2.90%**	273.0	**11.72%**	**0.691**
mMNLI	LeapAttack	1.80%	335.4	11.34%	0.532
	HyGloadAttack	1.80%	258.8	11.50%	0.676
	MVGText(ours)	**1.70%**	**257.0**	**11.21%**	**0.700**

C. Ablation Study

Global Fixed Word Replacement. The test success rate is directly related to the initialization and is highly random. To ensure fairness, we add a global fixed word replacement method that is independent of the random seed before the end of initialization. Compared to the classical algorithm HLGA, our method has the same or better test success rate, as shown in Table V. We also separately evaluated the global fixed word replacement strategy by counting the number of test cases generated by 49 synonym replacements, testing the three models on five datasets, and plotting line graphs according to index and times of occurrences, as shown in Fig. 3. We found that using only 49 synonym replacements yields a test success rate of over 80%. This indicates that the proposed strategy is effective. As the subscripts of the synonyms increase, the generated cases are more likely to be tested successfully. Using the first 10 synonym global replacements are less likely to be tested successfully compared to the last 10 synonyms. All 15 line graphs show this pattern.

Search for the New Test Case. The main improvement of MVGText is the search using multiple samples, which utilizes momentum and variance to guide the search for the new test case with lower perturbation rates. To evaluate the effectiveness of this process, we conduct an ablation study to remove the part of searching with multiple test cases and leave the other parts unchanged, which is called no_search. In order to reduce the computational overhead, we test the three models on only two classification datasets, MR and AG, and the experimental results are shown in Table VII. The results of the experiments test BERT on three NLI datasets and the results of the experiments are shown in Table VIII.

Greedy Optimization. Another improvement of MVGText is the use of greedy strategy to find test cases with higher similarity. We named the method optimized without using the greedy strategy no_greedy, and the experimental results on the classification dataset are shown in Table VII, and the experimental results on the NLI dataset are shown

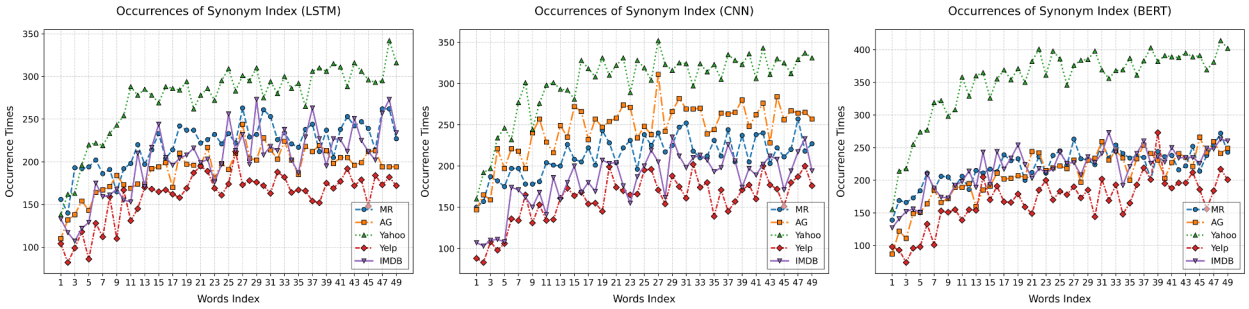

Fig. 3. Results of Global Fixed Word Replacement.

in Table VIII.

As can be seen from Table VII, MVGText obtains the lowest perturbation rate and also has a higher similarity compared to no_greedy. Although no_search has the highest similarity, the perturbation rate is too high and the generated test cases are more perceptible. The lower number of queries for no_search compared to no_greedy suggests that the search process utilizing multiple cases consumes a large number of queries. The test cases generated without applying the greedy optimization strategy have significantly lower similarity, which shows the effectiveness of the greedy optimization strategy. The experimental results in Table VIII also validate the above point that without using the search strategy, the test cases found have higher similarity but the perturbation rate is too high and the generated test cases are of low quality. Without applying the greedy strategy, although a lower perturbation is obtained, the similarity is also low and the quality is not high. MVGText that uses both search strategy and greedy strategy generates better quality test cases in comparison.

TABLE VII
ABLATION RESULTS OF MR AND AG.

Model	Dataset	Method	Acc↓	Qrs↓	Pert↓	Sim↑
LSTM	MR	no_search	0.70%	**203.4**	13.28%	**0.772**
		no_greedy	0.70%	353.7	10.40%	0.743
		MVGText	0.70%	375.8	**10.35%**	0.770
	AG	no_search	5.50%	**804.5**	17.06%	**0.807**
		no_greedy	5.50%	939.9	11.28%	0.713
		MVGText	5.50%	1100.7	**11.08%**	0.785
CNN	MR	no_search	0.60%	**202.1**	13.27%	**0.783**
		no_greedy	0.60%	360.9	10.15%	0.753
		MVGText	0.60%	383.7	**10.08%**	**0.783**
	AG	no_search	1.30%	**574.9**	12.91%	**0.862**
		no_greedy	1.30%	762	8.69%	0.796
		MVGText	1.30%	860.3	**8.50%**	0.845
BERT	MR	no_search	1.00%	**218**	13.16%	0.761
		no_greedy	1.00%	361.6	9.98%	0.735
		MVGText	1.00%	386	**9.94%**	**0.763**
	AG	no_search	2.00%	**786.4**	17.16%	**0.792**
		no_greedy	2.00%	882.8	10.75%	0.695
		MVGText	2.00%	1062.8	**10.59%**	0.763

VI. CONCLUSION

Studying hard-label text test case generation is important and challenging. Existing studies focused on optimizing one initial test case and may have missed better test cases. We propose MVGText, which calculates the momentum and based on multiple initial test cases to guide the search process. In the initialization phase, global fixed word replacement is performed to improve the test success rate. In the search phase, multiple initial test cases are used to find the test case with lower perturbations. Finally, a greedy strategy is used to find test cases with higher similarity. The results of the comparison experiments show that MVGText has better test success rate on most testing models, and achieves an average post-test success rate of 1.4%. Furthermore, MVGText has the highest similarity on eight datasets. Ablation studies show the effectiveness of the proposed improvements.

In the future, we will study how MVGText can be extended to test large language models and how MVGText performs in natural language generation tasks.

TABLE VIII
ABLATION RESULTS OF NLI.

Dataset	Method	Acc↓	Qrs↓	Pert↓	Sim↑
SNLI	no_search	1.10%	**112.2**	18.65%	**0.577**
	no_greedy	1.10%	161.3	15.82%	0.512
	MVGText	1.10%	176.1	**15.80%**	0.575
MNLI	no_search	2.90%	**189.9**	14.71%	**0.691**
	no_greedy	2.90%	258.3	**11.72%**	0.642
	MVGText	2.90%	273.0	**11.72%**	**0.691**
mMNLI	no_search	1.70%	**161.0**	14.32%	0.696
	no_greedy	1.70%	241.5	11.27%	0.654
	MVGText	1.70%	257.0	**11.21%**	**0.700**

ACKNOWLEDGMENT

The work is supported by the National Natural Science Foundation of China (Nos.62272145 and U21B2016).

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[2] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, L. Màrquez, C. Callison-Burch, and J. Su, Eds. Lisbon, Portugal: Association for Computational Linguistics, Sep. 2015, pp. 632–642.

[3] M. Zhang, Y.-j. Pei, G. Wang, and X.-x. Ma, "Large data recognition system for speech outliers based on deep learning," in *2021 International Conference on Big Data Analytics for Cyber-Physical System in Smart City: Volume 1*. Springer, 2022, pp. 915–925.

[4] S. Goyal, S. Doddapaneni, M. M. Khapra, and B. Ravindran, "A survey of adversarial defenses and robustness in nlp," *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–39, 2023.

[5] W. E. Zhang, Q. Z. Sheng, A. Alhazmi, and C. Li, "Adversarial attacks on deep-learning models in natural language processing: A survey," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 11, no. 3, pp. 1–41, 2020.

[6] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[7] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial intelligence safety and security*. Chapman and Hall/CRC, 2018, pp. 99–112.

[8] R. Maheshwary, S. Maheshwary, and V. Pudi, "Generating natural language attacks in a hard label black box setting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 15, 2021, pp. 13 525–13 533.

[9] M. Ye, J. Chen, C. Miao, T. Wang, and F. Ma, "Leapattack: Hard-label adversarial attack on text via gradient-based optimization," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2307–2315.

[10] L. Yuan, Y. Zhang, Y. Chen, and W. Wei, "Bridge the gap between cv and nlp! a gradient-based textual adversarial attack framework," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 7132–7146.

[11] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "Hotflip: White-box adversarial examples for text classification," *arXiv preprint arXiv:1712.06751*, 2017.

[12] H. Zhang, H. Zhou, N. Miao, and L. Li, "Generating fluent adversarial examples for natural languages," *arXiv preprint arXiv:2007.06174*, 2020.

[13] X. Han, Q. Li, H. Cao, L. Han, B. Wang, X. Bao, Y. Han, and W. Wang, "Bfs2adv: Black-box adversarial attack towards hard-to-attack short texts," *Computers & Security*, vol. 141, p. 103817, 2024.

[14] S. Ren, Y. Deng, K. He, and W. Che, "Generating natural language adversarial examples through probability weighted word saliency," in *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019, pp. 1085–1097.

[15] H. Peng, S. Guo, D. Zhao, X. Zhang, J. Han, S. Ji, X. Yang, and M. Zhong, "Textcheater: A query-efficient textual adversarial attack in the hard-label setting," *IEEE Transactions on Dependable and Secure Computing*, 2023.

[16] X. Wang and K. He, "Enhancing the transferability of adversarial attacks through variance tuning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 1924–1933.

[17] S. Samanta and S. Mehta, "Towards crafting text adversarial samples," *arXiv preprint arXiv:1707.02812*, 2017.

[18] M. Sato, J. Suzuki, H. Shindo, and Y. Matsumoto, "Interpretable adversarial perturbation in input embedding space for text," *arXiv preprint arXiv:1805.02917*, 2018.

[19] M. Behjati, S.-M. Moosavi-Dezfooli, M. S. Baghshah, and P. Frossard, "Universal adversarial attacks on text classifiers," in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 7345–7349.

[20] X. Zheng, J. Zeng, Y. Zhou, C.-J. Hsieh, M. Cheng, and X.-J. Huang, "Evaluating and enhancing the robustness of neural network-based dependency parsing models with adversarial examples," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 6600–6610.

[21] Z. Meng and R. Wattenhofer, "A geometry-inspired attack for generating natural language adversarial examples," *arXiv preprint arXiv:2010.01345*, 2020.

[22] J. Gao, J. Lanchantin, M. L. Soffa, and Y. Qi, "Black-box generation of adversarial text sequences to evade deep learning classifiers," in *2018 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2018, pp. 50–56.

[23] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is bert really robust? a strong baseline for natural language attack on text classification and entailment," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 8018–8025.

[24] G. Li, B. Shi, Z. Liu, D. Kong, Y. Wu, X. Zhang, L. Huang, and H. Lyu, "Adversarial text generation by search and learning," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 15 722–15 738.

[25] R. Jia and P. Liang, "Adversarial examples for evaluating reading comprehension systems," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 2021–2031.

[26] X. Xu, K. Kong, N. Liu, L. Cui, D. Wang, J. Zhang, and M. Kankanhalli, "An llm can fool itself: A prompt-based adversarial attack, 2023," *URL: http://arxiv. org/abs/2310.13345*.

[27] M. Ye, C. Miao, T. Wang, and F. Ma, "Texthoaxer: Budgeted hard-label adversarial attacks on text," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 4, 2022, pp. 3877–3884.

[28] H. Liu, Z. Xu, X. Zhang, X. Xu, F. Zhang, F. Ma, H. Chen, H. Yu, and X. Zhang, "Sspattack: a simple and sweet paradigm for black-box hard-label textual adversarial attack," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 228–13 235.

[29] M. Ye, J. Chen, C. Miao, H. Liu, T. Wang, and F. Ma, "Pat: Geometry-aware hard-label black-box adversarial attacks on text," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 3093–3104.

[30] Z. Liu, X. Xiong, Y. Li, Y. Yu, J. Lu, S. Zhang, and F. Xiong, "Hygloadattack: Hard-label black-box textual adversarial attacks via hybrid optimization," *Neural Networks*, p. 106461, 2024.

[31] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *stat*, vol. 1050, no. 9, 2017.

[32] N. Mrkšic, D. OSéaghdha, B. Thomson, M. Gašic, L. Rojas-Barahona, P.-H. Su, D. Vandyke, T.-H. Wen, and S. Young, "Counter-fitting word vectors to linguistic constraints," in *Proceedings of NAACL-HLT*, 2016, pp. 142–148.

[33] B. Pang and L. Lee, "Seeing stars: exploiting class relationships for sentiment categorization with respect to rating scales," in *Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics*, ser. ACL '05. USA: Association for Computational Linguistics, 2005, p. 115–124.

[34] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," *Advances in neural information processing systems*, vol. 28, 2015.

[35] A. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, 2011, pp. 142–150.

[36] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, M. Walker, H. Ji, and A. Stent, Eds. New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 1112–1122.

[37] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 11 1997.

[38] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," *Advances in neural information processing systems*, vol. 28, 2015.

[39] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.

[40] D. Cer, Y. Yang, S.-y. Kong, N. Hua, N. Limtiaco, R. St. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, B. Strope, and R. Kurzweil, "Universal sentence encoder for English," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, E. Blanco and W. Lu, Eds. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 169–174.